DML Commands

V P JAYACHITRA

DML OVERVIEW

- DML is used to retrieve, insert, update, and delete data in database tables
- DML statements query or manipulate data in existing schema objects.
- Unlike DDL, DML does not implicitly commit.

Key Characteristics

- Operates on **DATA (rows)**, not structure
- **Can be ROLLED BACK** before COMMIT
- Changes are temporary until COMMIT

DML vs DDL

Feature	DML	DDL
Works on	Data (rows)	Structure (schema)
Auto-commit?	NO	YES
Rollback?	YES	NO
Commands	INSERT, UPDATE, DELETE	CREATE, ALTER, DROP

INSERT STATEMENT

- INSERT ADDS A SINGLE TUPLE TO A RELATION.
- VALUES MUST MATCH THE ORDER OF ATTRIBUTES.

Syntax – Many Forms

- 1: INSERT with column list (BEST PRACTICE)

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

- 2: INSERT without column list

Must provide ALL columns in EXACT table order

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

- 3: Multi-row INSERT (Oracle 23c+)

```
INSERT INTO table_name (column1, column2)
VALUES
  (value1a, value2a),
  (value1b, value2b),
  (value1c, value2c);
```

Syntax – Many Forms

- 4: INSERT with DEFAULT

```
INSERT INTO table_name (column1, column2)
VALUES (value1, DEFAULT);
```

Column2 gets DEFAULT value

- 5 INSERT…SELECT (Bulk Insert)

```
INSERT INTO target_table (column1, column2)
SELECT column1, column2
FROM source_table
WHERE condition;
```

Data Type Rules

- Strings - Use single quotes
- Numbers - No quotes
- Dates - Three methods
  - DATE literal (YYYY-MM-DD)
  - TO_DATE function
  - SYSDATE

```
INSERT INTO EMP5001 (Empno, Empname)
VALUES (1001, 'Rajesh Kumar');
```

```
INSERT INTO EMP5001 (Empno, DoJ)
VALUES (1001, DATE '2024-01-15');

INSERT INTO EMP5001 (Empno, DoJ)
VALUES (1001, TO_DATE('15-01-2024', 'DD-MM-YYYY'));

INSERT INTO EMP5001 (Empno, DoJ)
VALUES (1001, SYSDATE);
```

NULL

Explicit NULL

```
INSERT INTO EMP (Empno, Empname, Commission)
VALUES (1001, 'Raj', NULL);
```

Implicit NULL (omit column)

```
INSERT INTO EMP (Empno, Empname)
VALUES (1001, 'Raj');
```

Commission gets NULL automatically

DEFAULT

Use DEFAULT keyword

```
INSERT INTO EMP (Empno, Empname, DoJ)
VALUES (1001, 'Raj', DEFAULT);
```

ignored column (gets DEFAULT if defined)

```
INSERT INTO EMP (Empno, Empname)
VALUES (1001, 'Raj');
```

If DoJ has DEFAULT SYSDATE, it gets current date

DEFAULT vs NULL

Column definition
Status VARCHAR2(10) DEFAULT 'ACTIVE'

Omit column → Gets DEFAULT
INSERT INTO Employee (Empno, Ename) VALUES (101, 'Raj');
Status = 'ACTIVE'

Explicit NULL → NULL overrides DEFAULT
INSERT INTO Employee (Empno, Ename, Status) VALUES (101, 'Raj', NULL);
Status = NULL (NOT 'ACTIVE')

DEFAULT keyword → Gets DEFAULT
INSERT INTO Employee (Empno, Ename, Status) VALUES (101, 'Raj', DEFAULT);
Status = 'ACTIVE'

Table structure

```
CREATE TABLE employee(
    Empno NUMBER(4) PRIMARY KEY,
    Empname VARCHAR2(50) NOT NULL,
    Deptno NUMBER(2) CHECK (Deptno BETWEEN 10 AND 90),
    Jobtitle VARCHAR2(50) CHECK (Jobtitle IN ('MANAGER','ANALYST','CLERK','SALESMAN')),
    DoJ DATE CHECK (DoJ <= SYSDATE),
    Salary NUMBER(8,2) CHECK (Salary > 0),
    Commission NUMBER(8,2) CHECK (Commission IS NULL OR Commission <= Salary)
);
```

INSERT-Examples

- Example 1: Complete INSERT with all columns

```
INSERT INTO EMP (Empno, Empname, Deptno, Jobtitle, DoJ, Salary, Commission)
VALUES (1001, 'Rajesh Kumar', 10, 'ANALYST', DATE '2024-01-15', 45000.00, NULL);
```

- Example 2: Column list in different order

```
INSERT INTO EMP (Empname, Salary, Empno, Deptno)
VALUES ('Priya Sharma', 50000.00, 1002, 20);
```

Order doesn't matter if column list provided

INSERT-Examples

- Example 3: Only required columns

```
INSERT INTO EMP (Empno, Empname, Salary)
VALUES (1003, 'Amit Patel', 35000.00);
Deptno, Jobtitle, DoJ, Commission get NULL
```

- Example 2: Without column list

```
INSERT INTO EMP
VALUES (1004, 'Sneha Reddy', 30, 'CLERK', DATE '2024-03-20', 28000.00, 2000.00);
 Must match exact table column order
```

INSERT-Examples

- With expressions

```
INSERT INTO EMP (Empno, Empname, Salary, Commission)
VALUES (1005, 'Vikram Singh', 40000, 40000 * 0.10);
Commission = 4000 (calculated)
```

- Multiple employees (Oracle 23c+)

```
INSERT INTO EMP (Empno, Empname, Deptno, Salary)
VALUES
  (1007, 'Ravi Kumar', 10, 35000),
  (1008, 'Divya Iyer', 20, 38000),
  (1009, 'Arjun Nair', 30, 32000);
```

INSERT...SELECT Examples

First create archive table

```
CREATE TABLE EMP_ARCHIVE AS SELECT * FROM EMP WHERE 1=0;
```

Copy old employees

```
INSERT INTO EMP_ARCHIVE
SELECT * FROM EMP
WHERE DoJ < DATE '2020-01-01';
```

Insert with aggregation

```
INSERT INTO DEPT_SUMMARY (Deptno, Emp_count, Total_salary)
SELECT Deptno, COUNT(*), SUM(Salary)
FROM EMP5001
GROUP BY Deptno;
```

Insert with JOIN

```
INSERT INTO EMP_DEPT_VIEW (Empno, Empname, Deptname)
SELECT E.Empno, E.Empname, D.Deptname
FROM EMP E JOIN DEPARTMENT D ON E.Deptno = D.Deptno;
```

Insert -Errors

- Error 1: ORA-00001 - Primary Key/Unique Violation
- **Error 2: ORA-01400 - NOT NULL Violation**
- Error 3: ORA-02290 - CHECK Constraint Violation
- Error 4: ORA-02291 - Foreign Key Violation
- Error 5: ORA-01722 - Invalid Number
- ORA-01858 - Date Format Error

```
INSERT INTO EMP5001 (Empno, Empname, Salary)
VALUES ('ABC', 'Test', 'Five Thousand');
ERROR: 'ABC' and 'Five Thousand' are not valid
```

```
INSERT INTO EMP5001 (Empno, Empname, DoJ, Salary)
VALUES (2006, 'Test', '2024-15-01', 30000);
ERROR: Invalid date format (month 15 doesn't exist)
Use: DATE '2024-01-15' or TO_DATE('15-01-2024', 'DD-MM-YYYY')
```

UPDATE STATEMENT

- UPDATE modifies attribute values of one or more selected tuples.
- A SET clause specifies attributes to modify and their new values.
- A WHERE clause selects tuples.
- Use UPDATE to change existing values in a table or in the base table of a view.

UPDATE STATEMENT

```
UPDATE table_name
SET column1 = value1,
    column2 = value2,
    ...
[WHERE condition];
```

UPDATE STATEMENT

- Update single column, single row

```
UPDATE table_name
SET column1 = value1
WHERE primary_key = value;
```

Example 1: Update single column, single employee

```
UPDATE EMP
SET Salary = 50000
WHERE Empno = 1001;
```

Only employee 1001's salary changed

- Update multiple columns, single row

```
UPDATE table_name
SET column1 = value1,
    column2 = value2,
    column3 = value3
WHERE primary_key = value;
```

Example 2: Update multiple columns

```
UPDATE EMP
SET Salary = 55000,
    Jobtitle = 'MANAGER',
    Commission = 5500
WHERE Empno = 1002;
```

UPDATE STATEMENT

- Update with expression

```
UPDATE table_name
SET column1 = column1 * 1.10,
    column2 = column1 + 1000
WHERE condition;;
```

Example 3: Update with calculation

```
UPDATE EMP5001
SET Salary = Salary * 1.10
WHERE Jobtitle = 'CLERK';
```

10% raise for all clerks

Example 4: Update based on multiple conditions

```
UPDATE EMP5001
SET Commission = Salary * 0.15
WHERE Jobtitle = 'SALESMAN' AND Deptno = 30;
```

Example 5: Update with CASE expression

```
UPDATE EMP5001
SET Salary = CASE
    WHEN Jobtitle = 'MANAGER' THEN Salary * 1.15
    WHEN Jobtitle = 'ANALYST' THEN Salary * 1.12
    WHEN Jobtitle = 'CLERK' THEN Salary * 1.08
    ELSE Salary * 1.05
END
WHERE Deptno = 10;
```

UPDATE STATEMENT

- Update with subquery

```
UPDATE table_name
SET column1 = (SELECT value FROM other_table WHERE condition)
WHERE condition;;
```

```
UPDATE EMP5001
SET Salary = (SELECT AVG(Salary) FROM EMP5001
WHERE Jobtitle = 'CLERK')
WHERE Empno = 1009 AND Jobtitle = 'CLERK';
```

UPDATE STATEMENT

- Update with CASE

```
UPDATE table_name
SET column1 = CASE
    WHEN condition1 THEN value1
    WHEN condition2 THEN value2
    ELSE value3
END
WHERE condition;
```

- Update all rows

```
UPDATE table_name
SET column1 = value1;
No WHERE clause = ALL rows affected
```

```
UPDATE EMP5001
SET Commission = 0;
 ALL employees get Commission = 0 (no WHERE clause)
```

DELETE STATEMENT

- DELETE removes tuples from a relation. WHERE clause selects tuples to delete. Without WHERE, all tuples are deleted
- Use DELETE to remove rows from a table, the base table of a view,

DELETE STATEMENT

- DELETE removes tuples from a relation. WHERE clause selects tuples to delete. Without WHERE, all tuples are deleted
- Use DELETE to remove rows from a table, the base table of a view,

```
DELETE FROM table_name
[WHERE condition];
```

DELETE STATEMENT

- Delete single row

```
DELETE FROM table_name
WHERE primary_key = value;
```

```
DELETE FROM EMP
WHERE Empno = 1001;
```

- Delete multiple rows

```
DELETE FROM table_name
WHERE condition;
```

```
DELETE FROM EMP
WHERE Deptno = 10;
```

Deletes all employees in department 10

```
DELETE FROM EMP
WHERE Jobtitle = 'CLERK' AND DoJ < DATE '2020-01-01';
```

DELETE STATEMENT

- Delete with subquery

```
DELETE FROM table_name
WHERE column IN (SELECT column FROM other_table WHERE condition);
```

```
DELETE FROM EMP
WHERE Deptno IN (SELECT Deptno FROM DEPARTMENT5001 WHERE Location = 'MUMBAI');
```

DELETE STATEMENT

- Delete all rows

DELETE FROM table_name;
No WHERE = ALL rows deleted (but can rollback)

DELETE FROM EMP5001;
Deletes ALL employees (can rollback)

Delete department (CASCADE effect)

PROJECT has FK with ON DELETE CASCADE

```
DELETE FROM DEPARTMENT
WHERE Deptno = 20;
```

Department deleted
All projects with Deptno=20 AUTOMATICALLY deleted

Delete department (SET NULL effect)

EMP might have FK with ON DELETE SET NULL

```
DELETE FROM DEPARTMENT
WHERE Deptno = 30;
```

Department deleted
All employees with Deptno=30 get Deptno=NULL

Which is BEST PRACTICE?

- A) INSERT INTO EMP5001 VALUES (1001, 'Raj', ...);
- B) INSERT INTO EMP5001 (Empno, Empname, ...) VALUES (1001, 'Raj', ...);

Which is BEST PRACTICE?

- A) INSERT INTO EMP5001 VALUES (1001, 'Raj', ...);
- B) INSERT INTO EMP5001 (Empno, Empname, ...) VALUES (1001, 'Raj', ...);

B - Explicit column list is maintainable. If table structure changes, B still works.

What happens?

INSERT INTO EMP (Empno, Empname, Salary) VALUES (2001, 'Ravi', 45000);

Deptno has CHECK (BETWEEN 10 AND 90). What is Deptno value?

A) Error: CHECK constraint violated
B) Deptno = 0
C) Deptno = NULL
D) Deptno = DEFAULT value

What happens?

INSERT INTO EMP5001 (Empno, Empname, Salary) VALUES (2001, 'Ravi', 45000);

Deptno has CHECK (BETWEEN 10 AND 90). What is Deptno value?

A) Error: CHECK constraint violated
B) Deptno = 0
C) Deptno = NULL
D) Deptno = DEFAULT value

C - CHECK constraint does NOT prevent NULL.

NULL satisfies CHECK (UNKNOWN passes).

Given: Status VARCHAR2(10) DEFAULT 'ACTIVE' What is inserted?

INSERT INTO Employee (Empno, Ename, Status) VALUES (101, 'Raj', NULL);

A) Status = 'ACTIVE'

B) Status = NULL

C) Error

Given: Status VARCHAR2(10) DEFAULT 'ACTIVE' What is inserted?

INSERT INTO Employee (Empno, Ename, Status) VALUES (101, 'Raj', NULL);

A) Status = 'ACTIVE'

B) Status = NULL

C) Error

B - Explicit NULL overrides DEFAULT. To get DEFAULT, omit column or use DEFAULT keyword.

What happens?

```
INSERT INTO EMP (Empno, Empname, Salary, Commission)
    VALUES (3001, 'Arjun', 50000, 60000);
```

CHECK: Commission IS NULL OR Commission <= Salary

A) Success

B) Error: Commission > Salary

What happens?

- INSERT INTO EMP5001 (Empno, Empname, Salary, Commission)

  VALUES (3001, 'Arjun', 50000, 60000);
- CHECK: Commission IS NULL OR Commission <= Salary
- A) Success

  B) Error: Commission > Salary

B - CHECK evaluates to FALSE (60000 > 50000), so row rejected.

What happens?

Oracle treats empty string ' ' as:

A) Empty string
B) NULL
C) Single space ' '
D) Error

What happens?

Oracle treats empty string '' as:

A) Empty string
B) NULL
C) Single space ' '
D) Error

B - Oracle treats '' as NULL (unique behavior).

What happens?

- UPDATE EMP5001

  SET Salary = Salary * 1.10;

A) Error: No WHERE clause
B) Only first row updated
C) All rows updated
D) No rows updated

What happens?

- UPDATE EMP5001

  SET Salary = Salary * 1.10;

A) Error: No WHERE clause
B) Only first row updated
C) All rows updated
D) No rows updated

C - Without WHERE, ALL rows affected. Very dangerous!

What is the result?

```
UPDATE EMP5001
SET Salary = 50000,
    Commission = Salary * 0.10
WHERE Empno = 1001;
```

What is Commission value?

A) 5000 (10% of 50000)
B) 10% of OLD salary
C) NULL
D) Error

What is the result?

```
UPDATE EMP5001
SET Salary = 50000,
    Commission = Salary * 0.10
WHERE Empno = 1001;
```

What is Commission value?

A) 5000 (10% of 50000)
B) 10% of OLD salary
C) NULL
D) Error

A - Commission uses NEW Salary value (50000) after SET.

Which UPDATE is INVALID?

A) UPDATE EMP5001 SET Salary = NULL WHERE Empno = 1001;

B) UPDATE EMP5001 SET Empno = 2001 WHERE Empno = 1001;

C) UPDATE EMP5001 SET Deptno = (SELECT Deptno FROM DEPARTMENT);

D) UPDATE EMP5001 SET Salary = Salary + 1000 WHERE Salary > 30000;

Which UPDATE is INVALID?

A) UPDATE EMP5001 SET Salary = NULL WHERE Empno = 1001;

B) UPDATE EMP5001 SET Empno = 2001 WHERE Empno = 1001;

C) UPDATE EMP5001 SET Deptno = (SELECT Deptno FROM DEPARTMENT);

D) UPDATE EMP5001 SET Salary = Salary + 1000 WHERE Salary > 30000;

C - Subquery must return single value.
If multiple departments exist, returns multiple rows = ERROR.

What is difference between DELETE and TRUNCATE?

A) DELETE is faster

B) TRUNCATE can be rolled back

C) DELETE can use WHERE, TRUNCATE cannot

D) Both are identical

What is difference between DELETE and TRUNCATE?

A) DELETE is faster

B) TRUNCATE can be rolled back

C) DELETE can use WHERE, TRUNCATE cannot

D) Both are identical

C - DELETE allows WHERE clause and can rollback.
TRUNCATE deletes all rows, no WHERE, cannot rollback.

What happens?

DELETE FROM DEPARTMENT WHERE Deptno = 10;

Given: PROJECT5001 has FK ON DELETE CASCADE, EMP5001 has FK ON DELETE SET NULL

A) Only department deleted
B) Error: child records exist
C) Department deleted, projects CASCADE deleted, employees' Deptno SET NULL
D) All related records deleted

What happens?

- DELETE FROM DEPARTMENT5001 WHERE Deptno = 10;
- Given: PROJECT5001 has FK ON DELETE CASCADE, EMP5001 has FK ON DELETE SET NULL
- A) Only department deleted
  B) Error: child records exist
  C) Department deleted, projects CASCADE deleted, employees' Deptno SET NULL
  D) All related records deleted

C - Different FK actions: Projects deleted (CASCADE), Employees keep existing but Deptno=NULL.